

Comparative analysis between Human Visual Search and Computer Vision Models

Midterm Report: Data Pipeline, Maths & Implementation

Cocoșilă-Dumitriu Rareș

Faculty of Mathematics and Computer Science

December 12, 2025

This presentation covers the full project status, structured as follows:

- ➊ **Introduction:** Project goals and the "Where's Waldo" paradigm.
- ➋ **Related Work:** Theoretical foundations (Human Vision) vs. SOTA Object Detection.
- ➌ **Data Acquisition:** Noaptea Cercetărilor & Hardware Setup.
- ➍ **Repository Structure:** Detailed breakdown of inputs, scripts, and generated outputs.
- ➎ **Processing Pipeline:** From raw files to metrics.
- ➏ **Mathematical Formalism:** Definitions of Saliency, Entropy, and Metrics.
- ➐ **Exploratory Data Analysis (EDA):** Visual heatmaps and behavioral statistics.
- ➑ **Models, Training & Compute:** Revision on YOLOv11 architecture, loss functions, and dynamics.
- ➒ **Future Directions:** AI gaze paths & advanced architectures.

Core Objective: Analyze the discrepancy between **Human search strategies** (Top-Down, Foveated, Serial) and **Computer vision models** (Bottom-Up, Parallel, Exhaustive) in high-entropy visual environments.

Why "Where's Waldo"?

- Represents the ultimate "Tiny Object Detection" challenge ($< 0.1\%$ pixel coverage).
- Forces humans to use complex strategies (scanning, inhibition of return) rather than simple "pop-out" detection.

Midterm Status Summary:

- **Data:** Pipeline fully operational (Raw $\rightarrow$ Clean $\rightarrow$ Features).
- **Analysis:** EDA completed; Distractors and Learning Effects identified.
- **Model:** Baseline YOLOv11 trained and evaluated with specific Loss definitions.

A. Biological Vision Theory

- **Feature Integration Theory (Treisman):**
"Waldo" lacks unique features, necessitating *serial search* (binding color + shape sequentially).
- **Yarbus (1967):** Eye movements are task-driven. The instruction "Find Waldo" creates a different scanpath than "Look at the beach".

B. Computational Models

- **Saliency Maps (Itti-Koch to DeepGaze):**
Algorithms that predict "eye-catching" regions based on contrast. We test if these apply to *active search*.
- **Object Detection (YOLO):** Typically trained on large objects (COCO). They lack the "temporal" aspect of search (saccades).

The Divergence

Humans use memory to avoid rescanning (Inhibition of Return). Standard AI models scan the entire image grid every inference, lacking temporal efficiency.

Ecological Validity: Data was collected "in the wild" during a public science event, ensuring diverse demographics compared to controlled lab studies.

Hardware Setup:

- **Device:** Neon Player Eye-Tracking Glasses (Pupil Labs).
- **Environment:** Uncontrolled lighting, variable distance to screen, high noise.
- **Artifacts:** Head movement compensation required (mapping gaze from "world camera" to "reference image").

Manual Ground Truth Generation:

- *Temporal:* Manual annotation of Start/Stop timestamps for each level.
- *Spatial:* Pixel-perfect coordinate mapping of Waldo using GIMP.

The project is organized to separate raw inputs from generated artifacts.

1. Raw Inputs (The Source):

- `recordings/` & `neon_player_fixations/`:
 - Directly exported from Eye-Tracker hardware.
 - Each subfolder = One participant session.
 - Contains: `gaze.csv`, `world.mp4`, `timestamps.csv`.
- `dataset/`:
 - The "Where's Waldo" High-Res Image Set.
 - Labels for YOLO training (Bounding Boxes).
 - Fetched via external storage (`wget waldo_training_images.tar`).

Our pipeline generates four distinct layers of data:

① `waldo_fixations/` (**Filtered Data**):

- Generated by: `waldo_fixations.py`.
- Content: Cleaned CSVs containing *only* relevant search fixations, categorized as Direct Hits or Peripheral Hits.

② `features/` (**Behavioral Metrics**):

- Generated by: `features_build.py`.
- Content: Statistical profiles per user (Saccade Length, Entropy, Revisits).

③ `attention_maps/` (**Visuals**):

- Generated by: `data_build.py`.
- Content: `.npy` (Raw numerical arrays) and `.png` (Gaussian Heatmaps).

④ `saliency_metrics/` (**Validation**):

- Generated by: `compute_saliency_metrics.py`.
- Content: Scores comparing Human Heatmaps vs. Computer Saliency.

Step 1: Filtering (`waldo_fixations.py`)

- *Challenge*: Raw data is a continuous stream.
- *Logic*: Apply manual timestamps → Filter I-VT (Velocity Threshold) → Check intersection with Waldo's Bounding Box (plus Foveal Margin).

Step 2: Feature Engineering (`features_build.py`) We transform coordinates (x, y, t) into behavioral insights:

- **Saccade Length**: Proxy for search speed.
- **Gaze Entropy**: High entropy = Random search; Low entropy = Systematic scanning.
- **Revisit Rate**: Measure of confusion (returning to empty spots).

Step 3: Visual Clustering (`data_build.py`)

- Applies **DBSCAN** to fixation points.
- **Goal**: Identify "Attractors" (Distractors) – clusters of fixations that are *not* Waldo.

To rigorously evaluate the alignment between Human Attention and Models, we employ:

A. Normalized Scanpath Saliency (NSS): Used to quantify how much saliency values diverge at human fixation points compared to the mean.

$$NSS(P, S) = \frac{1}{N} \sum_{i=1}^N \frac{S(x_i, y_i) - \mu_S}{\sigma_S} \quad (1)$$

Where $P = \{(x_i, y_i)\}$ is the set of human fixations, S is the Saliency Map, and N is total fixations. ($NSS > 1 \implies$ Strong Prediction).

B. Behavioral Entropy (Search Randomness): Quantifies the chaos in the search strategy over a discretized grid G :

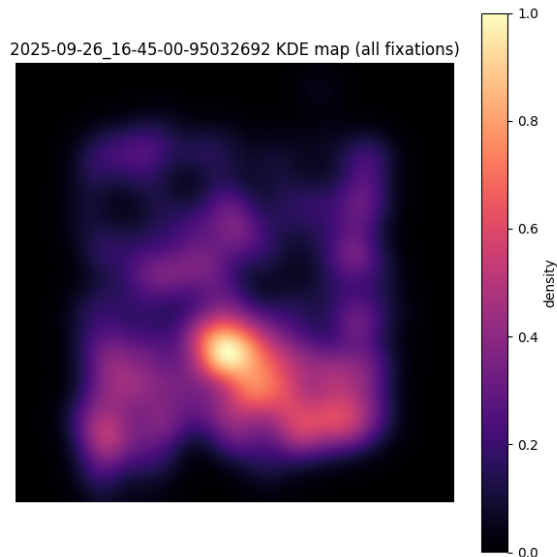
$$H(G) = - \sum_{k=1}^M p_k \log_2(p_k) \quad (2)$$

Where p_k is the probability of fixation falling into grid cell k . Low entropy implies a systematic "reading-like" strategy.

Visual Findings:

- **Center Bias:** Initial fixations tend towards the image center.
- **Distractor Clusters:** DBSCAN reveals that humans are consistently tricked by objects with **Color Congruency** (Red/White stripes) - e.g., Umbrellas, Sweaters.

This confirms Top-Down guidance: The instruction "Find Red Stripes" overrides general saliency.



Analysis of the aggregate `metrics_summary.csv` reveals:

① **The Learning Effect:**

$$T_{search}(Level_1) \gg T_{search}(Level_N)$$

Search time decreases drastically after the first level. Participants calibrate their internal filter for the specific artistic style and scale of Waldo.

② **No Return Bias (Efficiency):** Despite the chaos, humans rarely re-scan processed background areas. This demonstrates effective *Inhibition of Return* mechanism in working memory ($R_{rev} < 5\%$).

Baseline Model: YOLOv11s (Small) - Single-Stage Detector.

The Technical Challenge:

- Original Images: 1919×1079 pixels.
- Waldo Size: $\approx 50 \times 70$ pixels ($\approx 0.17\%$).
- If we resize (i.e. 640×640): Waldo may become < 1 pixel (Feature Loss).

Solution: Sliding Window Tiling

- We slice the high-res image into overlapping tiles (again, let's say 640×640).
- Run inference on each tile $\rightarrow$ Aggregate results $\rightarrow$ Non-Max Suppression (NMS).
- **Retry Mechanism:** If confidence is low, re-scan with different zoom/augmentation.

The model is trained minimizing a composite loss function $\mathcal{L}_{total}$:

$$\mathcal{L}_{total} = \lambda_{box} \mathcal{L}_{CIoU} + \lambda_{cls} \mathcal{L}_{BCE} + \lambda_{dfl} \mathcal{L}_{DFL} \quad (3)$$

- ① **Box Loss ($\mathcal{L}_{CIoU}$):** Complete-IoU Loss. Optimizes overlap, distance between centers, and aspect ratio consistency.
- ② **Class Loss ($\mathcal{L}_{BCE}$):** Binary Cross Entropy.

$$\mathcal{L}_{BCE} = - \sum (y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

- ③ **DFL (Distribution Focal Loss):** Refines bounding box boundaries when boundaries are ambiguous (crucial for tiny objects).

Evaluation Metric (mAP):

$$mAP = \frac{1}{|C|} \sum_{c \in C} \int_0^1 p_c(r) dr$$

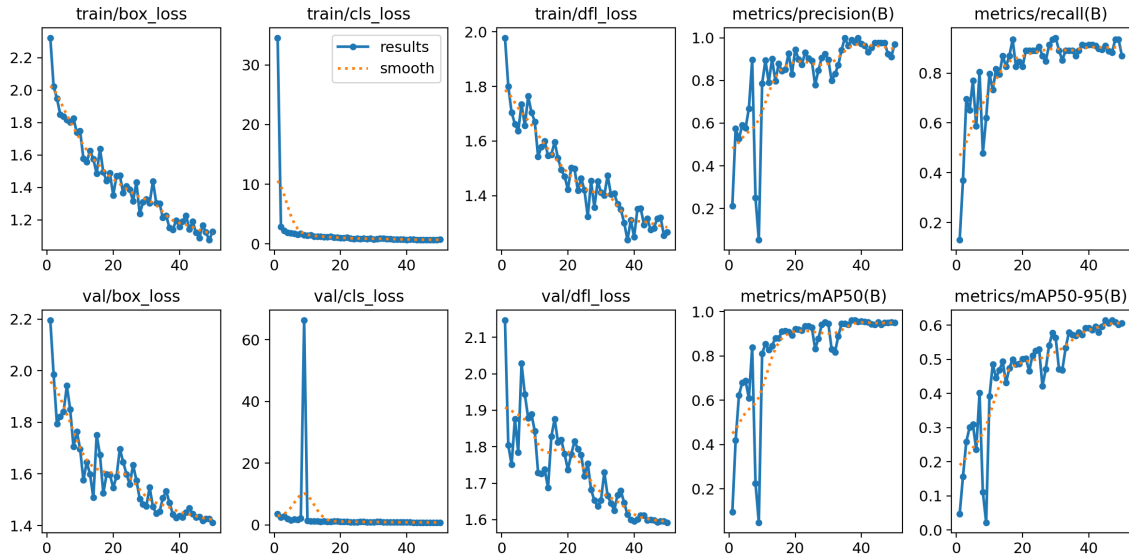
Hardware Specs (RTX 3050Ti laptop GPU):

- 4GB VRAM available; training ran reliably with batch size 2 and 2 dataloader workers.
- YOLO Small was used because it was fully sufficient for the task, while larger models would have trained somewhat slower and offered no meaningful advantage.
- Training ran in standard precision by default (FP32).

Training Observations:

- Box, cls and dfl losses drop consistently with no sign of collapse.
- Validation losses track training losses closely, so overfitting is negligible.
- Precision and recall ramp up fast and stabilize at high values by the end.
- mAP50 and mAP50-95 show steady monotonic improvement with no plateaus or regressions.
- More can be deduced from the figure on the next slide.

Training Curves



We employ a dual-metric system to evaluate both Attention and Detection.

1. Human-Saliency Alignment:

- **NSS Score:** Higher is better. Indicates AI saliency predicts human fixations.
- **AUC-Judd:** Measures classification ability (Fixated vs Non-Fixated).

2. Detection Performance:

- **mAP@50:** Mean Average Precision at 50% IoU threshold.
- **Search Efficiency?** Comparison of *Time to First Fixation (Human)* vs. *Inference Time (AI)*.

1. Alternative Architectures (Planned, but unnecessary?):

- **Faster R-CNN:** A two-stage detector (RPN + Classifier). Likely higher accuracy for tiny objects than YOLO, but slower.
- **Vision Transformers (ViT):** To capture global context (relations between objects) better than CNNs.

2. The dream: AI Gaze Path Generation

- *Goal:* Create a visual "Turing Test" for search strategies.
- *Method:* Record the sequence of tiles processed by the Sliding Window algorithm.
- *Metric:* Calculate Euclidean/Fréchet distance between Human Scanpath and AI Scanpath.

That's it, for now:

Repository: <https://github.com/Rares8921/Eye-Tracking-data-analysis>